SPRING BOOT

Application.properties

```
spring.application.name=security

server.port=5000

spring.security.user.name=fiit

spring.security.user.password=123
```

userController.java

```
package com.example.security.controller;

import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import jakarta.servlet.http.HttpServletRequest;


@RestController
@RequestMapping("/api")
public class userController {

    @GetMapping("/users")
    public String sayUser(){
        return "Hello User";
    }

    @GetMapping("/admin")
    public String sayAdmin(){
        return "Hello Admin";
    }

    @PostMapping("/access")
    public String addData(@RequestBody String data){
        return data;
    }

    @GetMapping("/csrf-token")
```

```java
    public CsrfToken getCsrfToken(HttpServletRequest request){
        return (CsrfToken) request.getAttribute("_csrf");
    }

    // csrf => token

    // /api/users

    // /api/admin
}
```

Theory Notes

1. Spring Security Basics

- **Spring Security** is a powerful framework for securing Java applications.
- By default, it enables:
  - **Authentication** → verifying user identity.
  - **Authorization** → controlling access to resources.
  - **CSRF protection** → preventing cross-site request forgery attacks.

2. Authentication

- In your `application.properties`, you defined:

  properties

  ```
  spring.security.user.name=fiit
  spring.security.user.password=123
  ```

- This sets up **basic authentication** with a default in-memory user.
- Requests to protected endpoints require credentials (`fiit:123`).

3. Authorization

- Endpoints can be restricted by roles (`ROLE_USER`, `ROLE_ADMIN`).
- Example:

  java

  ```
  @GetMapping("/users")    // accessible to USER role
  @GetMapping("/admin")    // accessible to ADMIN role
  ```

- Role-based access is configured in a **SecurityConfig** class using `HttpSecurity`.

4. CSRF Protection

- **CSRF (Cross-Site Request Forgery)** → malicious requests made on behalf of authenticated users.
- Spring Security generates a **CSRF token** for each session.
- Token must be included in state-changing requests (`POST`, `PUT`, `DELETE`).
- Example:

  java

  ```
  @GetMapping("/csrf-token")
  public CsrfToken getCsrfToken(HttpServletRequest request) {
      return (CsrfToken) request.getAttribute("_csrf");
  }
  ```

5. Key Concepts

- **Authentication** → who you are.
- **Authorization** → what you can do.
- **CSRF token** → ensures requests are genuine.
- **Security filters** → intercept requests before reaching controllers.

Coding Exercises

1. Create a `SecurityConfig` class to restrict `/api/admin` to users with `ROLE_ADMIN`.
2. Add a new endpoint `/api/public` that is accessible without authentication.
3. Implement CSRF token handling in a frontend form submission.